In [1]:
```
# This Python 3 environment comes with many helpful analytics libraries instal
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/d
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will lis

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that ge
# You can also write temporary files to /kaggle/temp/, but they won't be saved

# Use the kagglehub client library to attach Kaggle resources like competition
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/RE

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/s
ample_submission.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/t
rain.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/m
etadata.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/t
est.csv
```

In [2]:
```
import subprocess, sys
try:
    import catboost
except ImportError:
    subprocess.run([sys.executable, "-m", "pip", "install", "catboost", "-q"],

import numpy as np
import pandas as pd
import warnings
import os
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 60)
pd.set_option('display.float_format', '{:.2f}'.format)

from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error, r2_score
import lightgbm as lgb
from xgboost import XGBRegressor
```

```
from catboost import CatBoostRegressor

SEED = 42
np.random.seed(SEED)
print("ALL LIBRARIES LOADED OK.")
```

```
ALL LIBRARIES LOADED OK.
```

In [3]:
```
def rmsle(y_true, y_pred):
    y_pred = np.maximum(y_pred, 0)
    return np.sqrt(mean_squared_error(np.log1p(y_true), np.log1p(y_pred)))

def make_submission(preds, ids, filename='submission.csv'):
    preds = np.maximum(preds, 0)
    sub = pd.DataFrame({'TransactionID': ids, 'TargetValue': preds})
    sub.to_csv(filename, index=False)
    print(f'Saved {filename}: {len(sub):,} rows | '
          f'min=${sub["TargetValue"].min():,.0f} '
          f'median=${sub["TargetValue"].median():,.0f} '
          f'max=${sub["TargetValue"].max():,.0f}')
    return sub

print("Helper functions ready.")
```

```
Helper functions ready.
```

In [4]:
```
train_df = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pri
test_df  = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pri
sample   = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pri

print("=" * 55)
print(f"TRAIN : {train_df.shape[0]:>8,} rows x {train_df.shape[1]} cols")
print(f"TEST  : {test_df.shape[0]:>8,} rows x {test_df.shape[1]} cols")
print("=" * 55)
t = train_df['TargetValue']
print(f"Price range: ${t.min():,.0f} - ${t.max():,.0f}")
```

```
=======================================================
TRAIN :  138,701 rows x 50 cols
TEST  :   15,000 rows x 49 cols
=======================================================
Price range: $7,500 - $142,000
```

In [5]:
```
print("Building leak-free asset history features...")
full_df = pd.concat([train_df, test_df], axis=0, ignore_index=True)
# Sort by TransactionID to establish temporal order within each Asset
full_df = full_df.sort_values('TransactionID').reset_index(drop=True)

full_df['log_target'] = np.log1p(full_df['TargetValue'])
grouped = full_df.groupby('AssetID')

# Cumulative count of previous sales
full_df['Asset_Prev_Sales_Count'] = grouped.cumcount().astype(np.int32)

# Cumulative sum / count of target values BEFORE current row (shift ensures nc
```

```
full_df['cum_sum_log_target']      = grouped['log_target'].cumsum()
full_df['cum_sum_log_target_prev'] = full_df['cum_sum_log_target'] - full_df['

full_df['has_target']           = full_df['TargetValue'].notna().astype(np.int
full_df['cum_count_target']     = grouped['has_target'].cumsum()
full_df['cum_count_target_prev']= full_df['cum_count_target'] - full_df['has_t

# Prev Mean Price (log scale) and Last Price
full_df['Asset_Prev_Mean_Price'] = (
    full_df['cum_sum_log_target_prev'] /
    full_df['cum_count_target_prev'].replace(0, np.nan)
).astype(np.float32)
full_df['Asset_Prev_Last_Price'] = grouped['log_target'].shift(1).astype(np.fl

# Days since last sale for the same asset
full_df['TransactionDate_dt'] = pd.to_datetime(full_df['TransactionDate'], err
full_df['Asset_Days_Since_Last_Sale'] = grouped['TransactionDate_dt'].diff().d

# Drop helper columns
full_df = full_df.drop(columns=[
    'log_target', 'cum_sum_log_target', 'cum_sum_log_target_prev',
    'has_target', 'cum_count_target', 'cum_count_target_prev', 'TransactionDat
])

# Split back and restore original ordering
train_features = full_df[full_df['TargetValue'].notna()].copy()
test_features  = full_df[full_df['TargetValue'].isna()].copy()

train_features = train_features.set_index('TransactionID').loc[train_df['Trans
test_features  = test_features.set_index('TransactionID').loc[test_df['Transac

print(f"Asset history features added. Train: {train_features.shape} | Test: {t
```

```
Building leak-free asset history features...
Asset history features added. Train: (138701, 54) | Test: (15000, 54)
```

In [6]:
```
def preprocess_df(df):
    """
    BASE preprocessing only - no group-mean or frequency features.
    Those are computed fold-by-fold inside the CV loop to prevent leakage.
    """
    df = df.copy()

    # ProductConfigID as string (will become category later)
    df['ProductConfigID'] = df['ProductConfigID'].astype(str)

    # Date features
    df['TransactionDate'] = pd.to_datetime(df['TransactionDate'], errors='coer
    df['SaleYear']       = df['TransactionDate'].dt.year
    df['SaleMonth']      = df['TransactionDate'].dt.month
    df['SaleDay']        = df['TransactionDate'].dt.day
    df['SaleDayOfWeek']  = df['TransactionDate'].dt.dayofweek
    df['SaleDayOfYear']  = df['TransactionDate'].dt.dayofyear
    df['SaleQuarter']    = df['TransactionDate'].dt.quarter
```

```
    df['SaleWeek']       = df['TransactionDate'].dt.isocalendar().week.astype(

    # Time-based flags
    df['IsMonthEnd']    = df['TransactionDate'].dt.is_month_end.astype(np.int8
    df['IsMonthStart']  = df['TransactionDate'].dt.is_month_start.astype(np.in
    df['IsQuarterEnd']  = df['TransactionDate'].dt.is_quarter_end.astype(np.in
    df['IsQuarterStart']= df['TransactionDate'].dt.is_quarter_start.astype(np.
    df['IsYearEnd']     = df['TransactionDate'].dt.is_year_end.astype(np.int8)
    df['IsYearStart']   = df['TransactionDate'].dt.is_year_start.astype(np.int
    df = df.drop(columns=['TransactionDate'])

    # ManufactureYear cleaning
    df.loc[df['ManufactureYear'] <= 1920, 'ManufactureYear'] = np.nan

    # OperationalHoursMeter: 0 is placeholder for missing
    df.loc[df['OperationalHoursMeter'] == 0, 'OperationalHoursMeter'] = np.nan

    # Machine Age
    df['MachineAge'] = (df['SaleYear'] - df['ManufactureYear']).clip(lower=0)

    # Ordinal encodings (known natural order)
    tier_map  = {'Low': 0, 'Medium': 1, 'High': 2}
    scale_map = {'Mini': 0, 'Compact': 1, 'Small': 2, 'Medium': 3, 'Large / Me
    df['UtilizationTierOrd'] = df['UtilizationTier'].map(tier_map)
    df['AssetScaleOrd']      = df['AssetScaleFactor'].map(scale_map)

    # Hours engineering (clipping applied to training stats externally; small
    hours_99 = df['OperationalHoursMeter'].quantile(0.99)
    df['OperationalHoursMeter'] = df['OperationalHoursMeter'].clip(upper=hours
    df['HoursPresent'] = df['OperationalHoursMeter'].notna().astype(np.int8)
    df['LogHours']     = np.log1p(df['OperationalHoursMeter'].fillna(0))
    df['UsagePerYear'] = df['OperationalHoursMeter'] / (df['MachineAge'] + 1)

    # Age buckets
    df['IsNewMachine'] = (df['MachineAge'] <= 2).astype(np.int8)
    df['IsOldMachine'] = (df['MachineAge'] >= 20).astype(np.int8)
    df['AgeDecade']    = (df['MachineAge'] // 10).fillna(-1).astype(int)

    # Clean text and convert to category
    text_cols = df.select_dtypes(include='object').columns.tolist()
    df[text_cols] = df[text_cols].replace('None or Unspecified', np.nan)
    for col in text_cols:
        df[col] = df[col].fillna('Missing').astype('category')

    # Drop noisy columns
    for c in ['col18', 'col19', 'UtilizationTier', 'AssetScaleFactor']:
        if c in df.columns:
            df = df.drop(columns=[c])

    return df

print("Preprocessing features (no leaky group features)...")
train_proc = preprocess_df(train_features)
```

```
test_proc  = preprocess_df(test_features)
print(f"Train preprocessed: {train_proc.shape} | Test preprocessed: {test_proc
```

```
Preprocessing features (no leaky group features)...
Train preprocessed: (138701, 71) | Test preprocessed: (15000, 71)
```

In [7]:
```
ID_COLS = ['TransactionID', 'AssetID']

y_all        = np.log1p(train_proc['TargetValue'].values)
X_all        = train_proc.drop(columns=ID_COLS + ['TargetValue'], errors='ignc
X_test_final = test_proc.drop(columns=ID_COLS + ['TargetValue'], errors='ignor
test_ids     = test_df['TransactionID'].values

cat_cols = X_all.select_dtypes(include=['category', 'object']).columns.tolist(

print(f"Feature matrix : {X_all.shape}")
print(f"Test matrix    : {X_test_final.shape}")
print(f"Cat features   : {len(cat_cols)}")
print(f"Num features   : {X_all.shape[1] - len(cat_cols)}")
```

```
Feature matrix : (138701, 68)
Test matrix    : (15000, 68)
Cat features   : 40
Num features   : 28
```

In [8]:
```
# ── OOF Target Encoding ──────────────────────────────────────────────────
def add_oof_target_encoding(X_tr, y_tr, X_vl, X_te, cols, n_splits=5, alpha=15
    """
    Proper leak-free Out-Of-Fold target encoding.
    Training rows: encoded from the OTHER inner folds (never their own target)
    Validation / Test rows: encoded from the full training fold.
    """
    X_tr, X_vl, X_te = X_tr.copy(), X_vl.copy(), X_te.copy()
    global_mean = float(y_tr.mean())
    kf_inner = KFold(n_splits=n_splits, shuffle=True, random_state=seed)

    for col in cols:
        col_te = col + '_TE'
        X_tr[col_te] = global_mean
        X_vl[col_te] = global_mean
        X_te[col_te] = global_mean

        tr_val = X_tr[col].astype(str)
        vl_val = X_vl[col].astype(str)
        te_val = X_te[col].astype(str)

        # Map validation & test from full training fold stats
        stats = pd.DataFrame({'cat': tr_val, 'y': y_tr})
        agg   = stats.groupby('cat')['y'].agg(['mean', 'count'])
        agg['sm'] = (agg['count'] * agg['mean'] + alpha * global_mean) / (agg[
        m = agg['sm'].to_dict()
        X_vl[col_te] = vl_val.map(m).fillna(global_mean).values
        X_te[col_te] = te_val.map(m).fillna(global_mean).values
```

```
        # Map training rows from inner-fold stats
        col_idx = X_tr.columns.get_loc(col_te)
        for tr_in_idx, vl_in_idx in kf_inner.split(X_tr):
            in_stats = pd.DataFrame({'cat': tr_val.iloc[tr_in_idx], 'y': y_tr[
            in_agg   = in_stats.groupby('cat')['y'].agg(['mean', 'count'])
            in_agg['sm'] = (in_agg['count'] * in_agg['mean'] + alpha * global_
            m_in = in_agg['sm'].to_dict()
            X_tr.iloc[vl_in_idx, col_idx] = tr_val.iloc[vl_in_idx].map(m_in).f

    return X_tr, X_vl, X_te


# ── Fold-Level Group Statistics (frequency + relative features) ──────────
def add_fold_group_features(X_tr, X_vl, X_te):
    """
    Compute group-level statistics using ONLY the training fold, then map
    to validation and test. This prevents the distribution-mismatch bug
    that caused v12/v13 to score worse on Kaggle than locally.
    """
    X_tr, X_vl, X_te = X_tr.copy(), X_vl.copy(), X_te.copy()

    # 1. Frequency encoding for high-cardinality columns
    freq_cols = ['ProductConfigID', 'Spec_FullDescriptor', 'Spec_BaseClass', '
    for col in freq_cols:
        freq_map = X_tr[col].astype(str).value_counts().to_dict()
        global_freq = 1.0  # fallback for unseen categories
        for df_part, name in [(X_tr, 'tr'), (X_vl, 'vl'), (X_te, 'te')]:
            df_part[col + '_Freq'] = df_part[col].astype(str).map(freq_map).fi

    # 2. Category-level mean Hours and Age (grouped by InventoryGroupCategory)
    grp_col = 'InventoryGroupCategory'
    for target_col in ['OperationalHoursMeter', 'MachineAge']:
        global_mean = float(X_tr[target_col].mean())
        # Convert to str BEFORE groupby to avoid categorical dtype issues
        group_means = X_tr.groupby(X_tr[grp_col].astype(str))[target_col].mean
        feat_name   = f'Cat_{target_col}_mean'
        for df_part in [X_tr, X_vl, X_te]:
            df_part[feat_name] = df_part[grp_col].astype(str).map(group_means)
        # Relative feature: how does this machine compare to its category aver
        for df_part in [X_tr, X_vl, X_te]:
            df_part[f'{target_col}_rel_cat'] = (df_part[target_col] / (df_part

    # 3. Class and Config relative features
    for group_col in ['Spec_BaseClass', 'ProductConfigID']:
        for target_col in ['OperationalHoursMeter', 'MachineAge']:
            global_mean = float(X_tr[target_col].mean())
            # Convert to str BEFORE groupby to avoid categorical dtype issues
            group_means = X_tr.groupby(X_tr[group_col].astype(str))[target_col
            feat_name   = f'{group_col}_{target_col}_mean'
            for df_part in [X_tr, X_vl, X_te]:
                df_part[feat_name] = df_part[group_col].astype(str).map(group_
            for df_part in [X_tr, X_vl, X_te]:
                df_part[f'{target_col}_rel_by_{group_col}'] = (df_part[target_
```

```
    return X_tr, X_vl, X_te


TE_COLS = ['Spec_FullDescriptor', 'Spec_BaseClass', 'ProductConfigID', 'Functi
print("All fold-level feature engineering functions compiled. Ready for CV.")
```

```
All fold-level feature engineering functions compiled. Ready for CV.
```

In [9]:

```
N_FOLDS = 5
kf = KFold(n_splits=N_FOLDS, shuffle=True, random_state=SEED)

oof_lgb = np.zeros(len(X_all))
oof_xgb = np.zeros(len(X_all))
oof_cat = np.zeros(len(X_all))

test_lgb_folds = np.zeros((len(X_test_final), N_FOLDS))
test_xgb_folds = np.zeros((len(X_test_final), N_FOLDS))
test_cat_folds = np.zeros((len(X_test_final), N_FOLDS))

fold_scores = []

print("Starting 5-Fold CV — v14 (Leak-Fixed Ensemble)")
print("=" * 70)

for fold, (tr_idx, vl_idx) in enumerate(kf.split(X_all), 1):
    X_tr_raw = X_all.iloc[tr_idx].copy()
    X_vl_raw = X_all.iloc[vl_idx].copy()
    y_tr     = y_all[tr_idx]
    y_vl     = y_all[vl_idx]

    # Step 1: OOF Target Encoding (leak-free, 4 columns)
    X_tr, X_vl, X_te = add_oof_target_encoding(
        X_tr_raw, y_tr, X_vl_raw, X_test_final.copy(), TE_COLS, alpha=15.0
    )

    # Step 2: All group/frequency features (computed from training fold only)
    X_tr, X_vl, X_te = add_fold_group_features(X_tr, X_vl, X_te)

    # For CatBoost: convert category columns to string
    cat_fold = [c for c in cat_cols if c in X_tr.columns]

    def to_str(df, cols):
        df = df.copy()
        for c in cols:
            if c in df.columns:
                df[c] = df[c].astype(str)
        return df

    X_tr_c = to_str(X_tr, cat_fold)
    X_vl_c = to_str(X_vl, cat_fold)
    X_te_c = to_str(X_te, cat_fold)

    # ── LightGBM ──────────────────────────────────────────────────────
```

```
    lgb_m = lgb.LGBMRegressor(
        n_estimators      = 10000,
        max_depth         = -1,
        num_leaves        = 255,
        learning_rate     = 0.015,
        subsample         = 0.75,
        colsample_bytree  = 0.75,
        min_child_samples = 20,
        reg_alpha         = 0.1,
        reg_lambda        = 2.0,
        random_state      = SEED,
        n_jobs            = -1,
        verbose           = -1
    )
    lgb_m.fit(
        X_tr, y_tr,
        eval_set=[(X_vl, y_vl)],
        callbacks=[lgb.early_stopping(250, verbose=False), lgb.log_evaluation(
    )
    oof_lgb[vl_idx]          = lgb_m.predict(X_vl)
    test_lgb_folds[:, fold-1] = lgb_m.predict(X_te)

    # ── XGBoost ──────────────────────────────────────────────────────────
    xgb_m = XGBRegressor(
        n_estimators          = 10000,
        max_depth             = 9,
        learning_rate         = 0.015,
        subsample             = 0.75,
        colsample_bytree      = 0.75,
        tree_method           = 'hist',
        enable_categorical    = True,
        min_child_weight      = 5,
        gamma                 = 0.1,
        reg_alpha             = 0.1,
        reg_lambda            = 2.0,
        random_state          = SEED,
        n_jobs                = -1,
        early_stopping_rounds = 250
    )
    xgb_m.fit(X_tr, y_tr, eval_set=[(X_vl, y_vl)], verbose=False)
    oof_xgb[vl_idx]          = xgb_m.predict(X_vl)
    test_xgb_folds[:, fold-1] = xgb_m.predict(X_te)

    # ── CatBoost ─────────────────────────────────────────────────────────
    cat_m = CatBoostRegressor(
        iterations            = 10000,
        learning_rate         = 0.015,
        random_seed           = SEED,
        verbose               = 0,
        thread_count          = -1,
        early_stopping_rounds = 250,
        cat_features          = cat_fold,
        l2_leaf_reg           = 3.0,
```

```
        bagging_temperature   = 0.5,
        grow_policy           = 'Lossguide',
        max_leaves            = 255,
        min_data_in_leaf      = 20
    )
    cat_m.fit(X_tr_c, y_tr, eval_set=(X_vl_c, y_vl))
    oof_cat[vl_idx]          = cat_m.predict(X_vl_c)
    test_cat_folds[:, fold-1] = cat_m.predict(X_te_c)

    s_l = rmsle(np.expm1(y_vl), np.expm1(oof_lgb[vl_idx]))
    s_x = rmsle(np.expm1(y_vl), np.expm1(oof_xgb[vl_idx]))
    s_c = rmsle(np.expm1(y_vl), np.expm1(oof_cat[vl_idx]))
    fold_scores.append((s_l, s_x, s_c))

    print(f"Fold {fold}: LGB={s_l:.5f}(iter={lgb_m.best_iteration_:4d}) | "
          f"XGB={s_x:.5f}(iter={xgb_m.best_iteration:4d}) | "
          f"CAT={s_c:.5f}")

oof_l = rmsle(np.expm1(y_all), np.expm1(oof_lgb))
oof_x = rmsle(np.expm1(y_all), np.expm1(oof_xgb))
oof_c = rmsle(np.expm1(y_all), np.expm1(oof_cat))
print("=" * 70)
print(f"LGB OOF RMSLE : {oof_l:.5f}  (v10 Kaggle=0.19020, target=<0.19020)")
print(f"XGB OOF RMSLE : {oof_x:.5f}")
print(f"CAT OOF RMSLE : {oof_c:.5f}")
```

```
Starting 5-Fold CV — v14 (Leak-Fixed Ensemble)
======================================================================
Fold 1: LGB=0.19539(iter=5429) | XGB=0.20133(iter=7041) | CAT=0.20421
Fold 2: LGB=0.19796(iter=5591) | XGB=0.20334(iter=6421) | CAT=0.20697
Fold 3: LGB=0.19847(iter=4365) | XGB=0.20421(iter=5331) | CAT=0.20656
Fold 4: LGB=0.19777(iter=4753) | XGB=0.20443(iter=7527) | CAT=0.20835
Fold 5: LGB=0.19705(iter=3818) | XGB=0.20294(iter=6041) | CAT=0.20481
======================================================================
LGB OOF RMSLE : 0.19733  (v10 Kaggle=0.19020, target=<0.19020)
XGB OOF RMSLE : 0.20325
CAT OOF RMSLE : 0.20619
```

In [10]:

```
print("Searching optimal blend weights...")
best_blend, best_w = 999.0, (0.75, 0.0, 0.25)

for w1 in np.arange(0.0, 1.01, 0.05):
    for w2 in np.arange(0.0, 1.01 - w1, 0.05):
        w3    = max(round(1.0 - w1 - w2, 6), 0.0)
        score = rmsle(np.expm1(y_all),
                      np.expm1(w1*oof_lgb + w2*oof_xgb + w3*oof_cat))
        if score < best_blend:
            best_blend, best_w = score, (w1, w2, w3)

print("=" * 70)
print(f"Best blend    : LGB={best_w[0]:.2f}  XGB={best_w[1]:.2f}  CAT={best_w[
print(f"Blended RMSLE : {best_blend:.5f}  (v10 local was 0.19604)")
target = 0.19020
status = "[ON TRACK] Beat 0.19020!" if best_blend < target else "[KEEP GOING]
```

```
print(f"Kaggle target : {target}  ->  {status}")
```

```
Searching optimal blend weights...
======================================================================
Best blend    : LGB=0.70  XGB=0.00  CAT=0.30
Blended RMSLE : 0.19542  (v10 local was 0.19604)
Kaggle target : 0.1902  ->  [KEEP GOING] Still above 0.19020...
```

In [11]:
```
w1, w2, w3 = best_w

test_blend   = w1*test_lgb_folds.mean(1) + w2*test_xgb_folds.mean(1) + w3*test
final_prices = np.expm1(test_blend)

# Clip to training price range (0.1th–99.9th percentile)
plo = train_df['TargetValue'].quantile(0.001)
phi = train_df['TargetValue'].quantile(0.999)
final_prices = np.clip(final_prices, plo, phi)
print(f"Price clipped to [${plo:,.0f}, ${phi:,.0f}]")

sub = make_submission(final_prices, test_ids, 'submission.csv')
print()
print(sub.head())
```

```
Price clipped to [$7,500, $137,500]
Saved submission.csv: 15,000 rows | min=$7,500 median=$34,600 max=$137,500

   TransactionID  TargetValue
0        1139307     70033.18
1        1139419     81953.00
2        1139482     32062.96
3        1139522     20163.85
4        1139684     12830.27
```

In [ ]:
```
oof_blend  = best_w[0]*oof_lgb + best_w[1]*oof_xgb + best_w[2]*oof_cat
pred_price = np.expm1(oof_blend)
true_price = np.expm1(y_all)
residuals  = true_price - pred_price
pct_err    = (residuals / true_price) * 100

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
fig.suptitle(f'v14 OOF Diagnostics (Leak-Fixed) | Blended RMSLE={best_blend:.5
             fontsize=12, fontweight='bold')

idx = np.random.choice(len(true_price), min(5000, len(true_price)), replace=Fa
axes[0].scatter(true_price[idx], pred_price[idx], alpha=0.07, s=4, color='stee
mn, mx = true_price.min(), true_price.max()
axes[0].plot([mn, mx], [mn, mx], 'r--', lw=2, label='Perfect')
axes[0].set_xlabel('Actual Price');  axes[0].set_ylabel('Predicted')
axes[0].set_title('Actual vs Predicted'); axes[0].legend(fontsize=9)
fmt = lambda x, _: f'${x/1000:.0f}K'
axes[0].xaxis.set_major_formatter(mticker.FuncFormatter(fmt))
axes[0].yaxis.set_major_formatter(mticker.FuncFormatter(fmt))

axes[1].hist(residuals.clip(-15000, 15000), bins=80, color='coral', edgecolor=
```

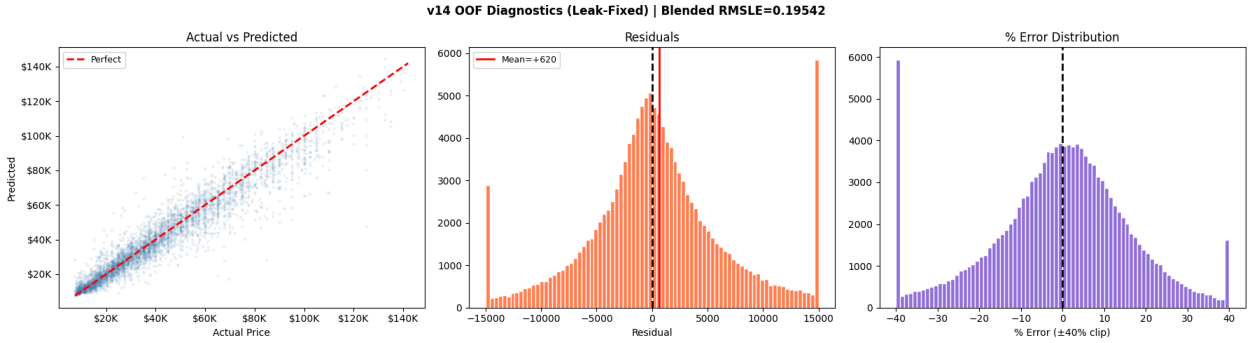
```
axes[1].axvline(0, color='black', lw=2, ls='--')
axes[1].axvline(residuals.mean(), color='red', lw=2, label=f'Mean={residuals.m
axes[1].set_xlabel('Residual'); axes[1].set_title('Residuals'); axes[1].legend

axes[2].hist(pct_err.clip(-40, 40), bins=80, color='mediumpurple', edgecolor='
axes[2].axvline(0, color='black', lw=2, ls='--')
axes[2].set_xlabel('% Error (±40% clip)'); axes[2].set_title('% Error Distribu

plt.tight_layout(); plt.show()

print(f"Blended OOF RMSLE : {best_blend:.5f}")
print(f"R2                : {r2_score(true_price, pred_price):.4f}")
print(f"MAE               : ${np.mean(np.abs(residuals)):,.0f}")
print(f"Median % error    : {np.median(np.abs(pct_err)):.1f}%")

# Feature importances from last LGB fold
fi = pd.Series(lgb_m.feature_importances_, index=X_tr.columns).sort_values(asc
print("\nTop 25 features (LGB last fold):")
print(fi.head(25).to_string())ou90
```



```
Blended OOF RMSLE : 0.19542
R2                : 0.9140
MAE               : $5,135
Median % error    : 10.0%

Top 25 features (LGB last fold):
SaleDayOfYear                                    70431
MachineAge_rel_by_ProductConfigID                63347
RegionCode                                       59957
SaleDay                                          56805
MachineAge_rel_by_Spec_BaseClass                 51454
ProductConfigID                                  46598
SaleYear                                         39583
ManufactureYear                                  35248
MachineAge_rel_cat                               30945
Spec_BaseClass                                   29656
FunctionalClassification_TE                      29470
Spec_FullDescriptor_TE                           28692
Spec_BaseClass_TE                                27096
Spec_FullDescriptor                              25292
MachineAge                                       21973
SaleDayOfWeek                                    20705
ProductConfigID_Freq                             19869
OperationalHoursMeter_rel_by_ProductConfigID     18870
FunctionalClassification                         18395
SaleWeek                                         17277
OperationalHoursMeter_rel_by_Spec_BaseClass      16941
UsagePerYear                                     16020
VendorPartnerID_Freq                             14456
OperationalHoursMeter                            14305
VendorPartnerID                                  14091
```

In [ ]: